

Course: Individual Software Development Process' 2026

Software Requirement Specification

By TripleT

Chosen Irl Challenge: Project A: Lab Data Management



Archan Norsiri	6810545981
Pensiri Yakongko	6810545778
Tanisorn Pisittanaphat	6810545671
Tanawat Rungwallapa	6810545646
Panuwitch Sowkasem	6810545867

Table of Contents

1. Target Users.....	1
2. The System Goal.....	1
3. Goal Refinement via KAOS:.....	2
4. Use Case Diagram.....	2
5. User Stories.....	3
6. User Requirement SpecificationZ.....	5
7. Activity Diagram.....	6
AD-1: Lab Member Books a Resource.....	6
AD-2: Lab Member Views and Manages Bookings.....	7
AD-3: Lab Staff Reports and Administrator Resolves a Resource Problem.....	8
AD-4: Administrator Manages Lab Resources.....	9
AD-5: Administrator Reviews a Pending Booking Request.....	10
AD-6: Administrator Manages Roles and Permissions.....	11
8. System Requirement Specification.....	12
9. Software Architecture.....	13
10. Data Storage Technology.....	15
11. Development Environment.....	16
12. Sequence Diagrams for All SRS.....	17
SQD-1: Lab Member Views Lab Resources.....	17
SQD-2: Lab Member Finds Resources (Search / Filter / Sort).....	18
SQD-3: Lab Member Submits Booking Request - Success Path.....	19
SQD-4: Lab Member Submits Booking Request - Conflict / Unavailable Path.....	20
SQD-5: Lab Member Views and Cancels Own Booking.....	21
SQD-6: Administrator Approves or Rejects Pending Booking.....	22
SQD-7: Lab Staff Reports a Resource Problem.....	23
SQD-8: Administrator Resolves an Issue Report.....	24
SQD-9: Administrator Creates / Edits / Sets Availability / Archives a Resource.....	25
SQD-10: Administrator Resource Update - Validation Error Path.....	26
SQD-11: Administrator Assigns or Revokes User Role.....	27
SQD-12: Administrator Views Change History.....	28
SQD-13: User Signs In with Google OAuth 2.0.....	29
SQD-14: Administrator Views Administrative Records.....	30
13. Traceability Matrix.....	31

1. Target Users

- **Lab Member (Ph.D. students, undergraduate students, and professors):** Sign in, view and find resources, view resource details, submit booking requests, view own bookings/requests, and cancel an eligible own booking
- **Lab Staff (Lab managers or operational laboratory staff when reporting operational/data issues):** Sign in and report resource problems or data-correction issues so that an authorised administrator can review and resolve them
- **System Administrator (Authorised administrative role, which may be assigned to an appropriate lab manager):** Sign in, manage resource records and availability, review booking requests, resolve issue reports, manage user roles and permissions, view change history, and view administrative records

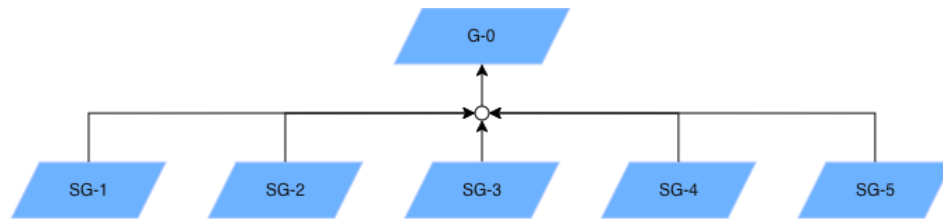
2. The System Goal

G-0: Maintain accurate and traceable lab data in one central system.

The lab currently keeps operational data in separate tools and manual records, which can lead to inconsistent, duplicated, or difficult-to-audit information. This iteration centralises resource management, booking, issue reporting, role-based access, authentication, administrative records, and change history for the three system actors defined above.

Reports and data export, budgeting, integrations with external operational systems, and production deployment infrastructure are outside the scope of this iteration. Docker containerisation is included only to provide a consistent development/testing runtime and deployment packaging basis.

3. Goal Refinement via KAOS:



SG-1 - Achieve: Provide lab members with resource visibility and reliable booking.

Rationale: Lab members need to know which resources exist and reserve them without clashing with other members' approved usage.

SG-2 - Maintain: Accurate and consistent lab records.

Rationale: Resource, booking, issue-report, and operational data should follow consistent formats to reduce duplicate, missing, or incorrect information.

SG-3 - Achieve: Allow authorised users to manage lab data.

Rationale: Administrative data-management actions should remain restricted to authorised administrators while normal users receive only the functions required by their role.

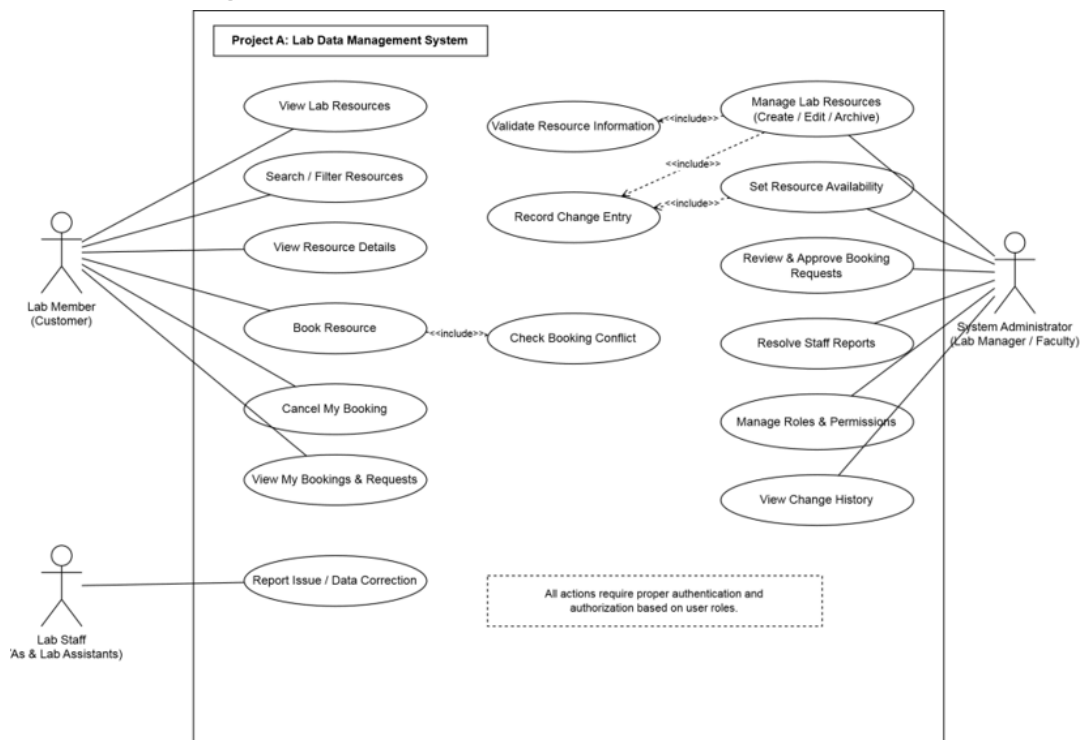
SG-4 - Avoid: Unauthorised or accidental modification of lab data.

Rationale: Google OAuth 2.0 authentication and role-based access control protect restricted functions and data from unauthorised modification.

SG-5 - Maintain: Traceable and auditable lab records.

Rationale: Important changes should record the acting user, timestamp, action, and affected record so that administrators can review what changed and when.

4. Use Case Diagram



5. User Stories

User Story ID	Description	Acceptance Criteria
US-1	As a lab member, I want to view lab resources so that I can know what resources are available for use.	The system displays current non-archived resources.
US-2	As a lab member, I want to find lab resources so that I can quickly narrow and organise the resource list.	The resource list supports search by name, filtering by category and availability status, and sorting by name, category, or availability status.
US-3	As a lab member, I want to view resource details so that I can decide whether a resource is suitable for my work.	The selected resource shows description, location, responsible person, availability, and current status.
US-4	As a lab member, I want to submit a booking request so that I can use a resource during a required time period.	A booking request is created only when the required booking data is valid and the resource is available for the requested period.
US-5	As a lab member, I want to view my bookings and requests so that I can track my planned resource usage.	The system shows only the lab member's own bookings and requests with their current status.
US-6	As a lab member, I want to cancel my booking so that I can release a time period I no longer need.	A lab member can cancel only their own booking before its start time, and the reserved period is released.
US-7	As a lab staff member, I want to report a resource problem so that the administrator can correct the central record or arrange maintenance.	The report stores the affected resource, problem type, description, reporter, submission time, and Pending status, and the administrator is notified.
US-8	As a system administrator, I want to create a resource record so that a new lab resource can be managed in the system.	An authorised administrator can create a valid resource record.
US-9	As a system administrator, I want to edit a resource record so that lab information remains accurate.	An authorised administrator can save valid changes to an existing resource record.
US-10	As a system administrator, I want to archive a resource record so that retired resources are hidden from normal lab-member views.	An authorised administrator can archive a resource; archived resources remain visible in the administrator view but not in the normal member list.
US-11	As a system administrator, I want to set resource availability so that members can see whether a resource can currently be booked.	An authorised administrator can change the availability status of a resource.
US-12	As a system administrator, I want to review a pending booking request so that I can approve valid requests or reject unsuitable ones.	The administrator can approve or reject a pending request; approval reserves the requested period, rejection records a reason, and the requester is notified of the decision.

User Story ID	Description	Acceptance Criteria
US-13	As a system administrator, I want to resolve an issue report so that the reported resource problem has a recorded outcome.	An authorised administrator can update the report status, add a resolution note, and notify the reporter.
US-14	As a system administrator, I want to manage a user role so that each user has permissions appropriate to their responsibilities.	An authorised administrator can assign or revoke a role, and the corresponding permissions take effect immediately after the change is saved.
US-15	As a system administrator, I want to view change history so that I can audit who changed what and when.	The history shows the acting user, affected record, action, and timestamp in chronological order.
US-16	As a user, I want to sign in using Google OAuth 2.0 so that I can securely access functions permitted by my role.	A user can sign in with Google OAuth 2.0; unauthenticated users cannot access protected functions.
US-17	As a system administrator, I want to view administrative records so that I can oversee laboratory operations from one place.	An authorised administrator can view resource, booking, issue-report, user-role, and audit-log records relevant to lab operations.

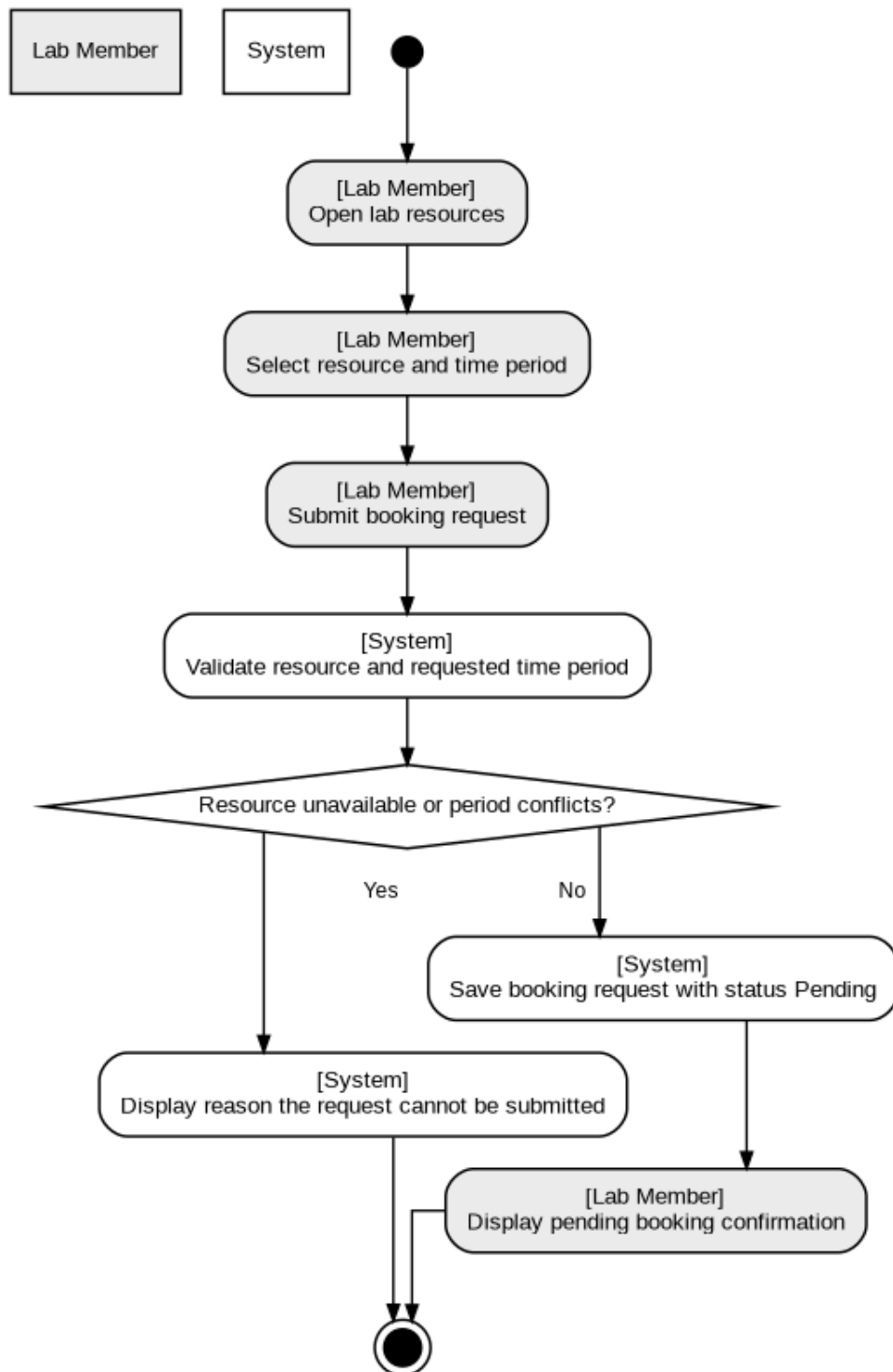
6. User Requirement SpecificationZ

ID	User Requirement
URS-1	Lab members shall be able to browse current non-archived lab resources and find relevant resources using search, filtering, and sorting.
URS-2	Lab members shall be able to view the details of a selected lab resource.
URS-3	Lab members shall be able to submit a booking request for a specified time period, and the system shall prevent unavailable or conflicting bookings from being accepted.
URS-4	Lab members shall be able to view their own bookings and requests and cancel an eligible own booking before its start time.
URS-5	Lab staff shall be able to report a resource problem or data-correction issue.
URS-6	An authorised administrator shall be able to manage resource records, including creating, editing, archiving, and setting availability.
URS-7	An authorised administrator shall be able to review pending booking requests and approve or reject them.
URS-8	An authorised administrator shall be able to resolve issue reports from lab staff.
URS-9	An authorised administrator shall be able to manage user roles and permissions by assigning or revoking roles.
URS-10	An authorised administrator shall be able to audit changes through a chronological history of resource, booking, issue-report, and role changes.
URS-11	Users shall be able to securely sign in using Google OAuth 2.0 and access only functions permitted by their assigned role.
URS-12	An authorised administrator shall be able to view administrative records for resources, bookings, issue reports, user roles, and audit logs.

7. Activity Diagram

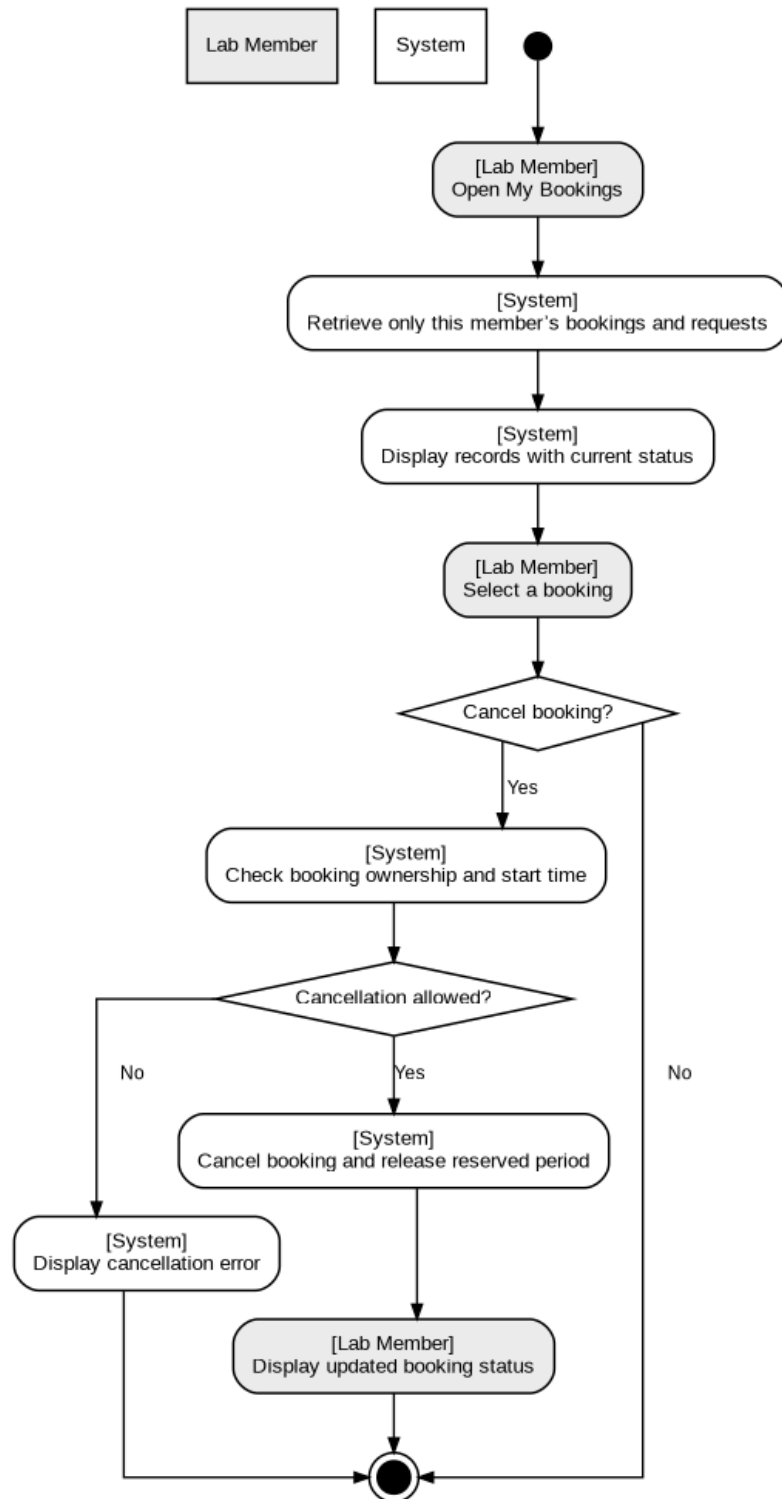
AD-1: Lab Member Books a Resource

Traced requirements: SRS-1, SRS-3, and SRS-4.



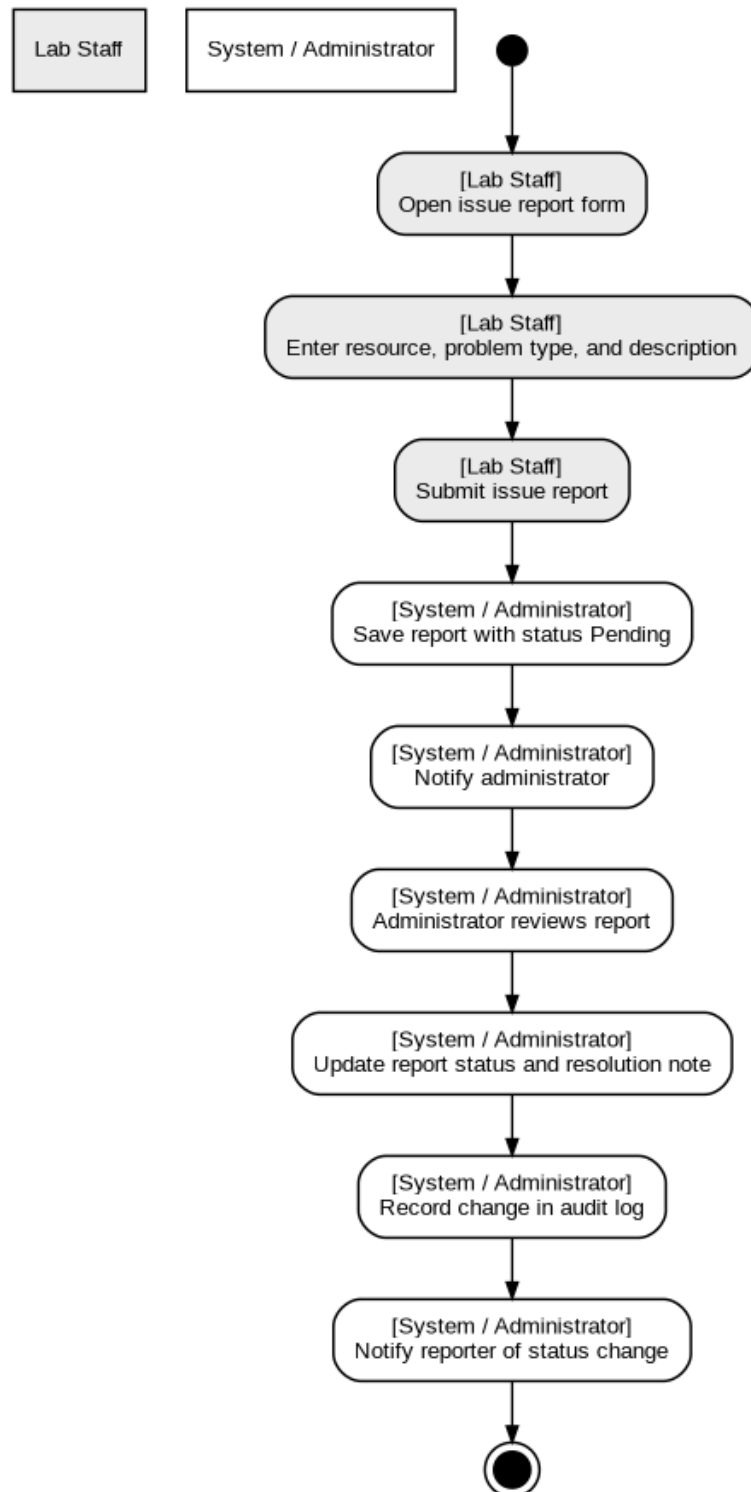
AD-2: Lab Member Views and Manages Bookings

Traced requirements: SRS-5.



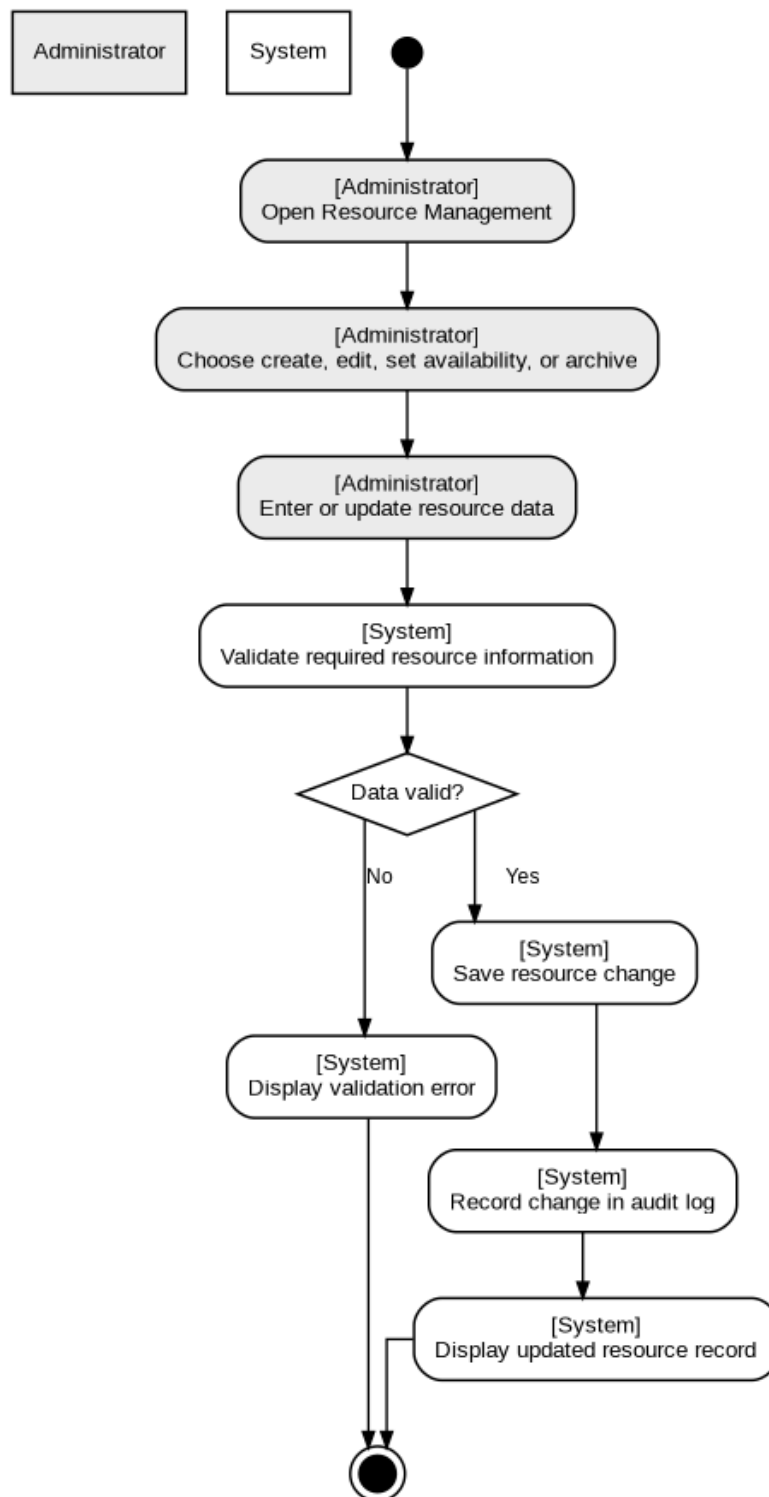
AD-3: Lab Staff Reports and Administrator Resolves a Resource Problem

Traced requirements: SRS-8, SRS-9, and SRS-13.



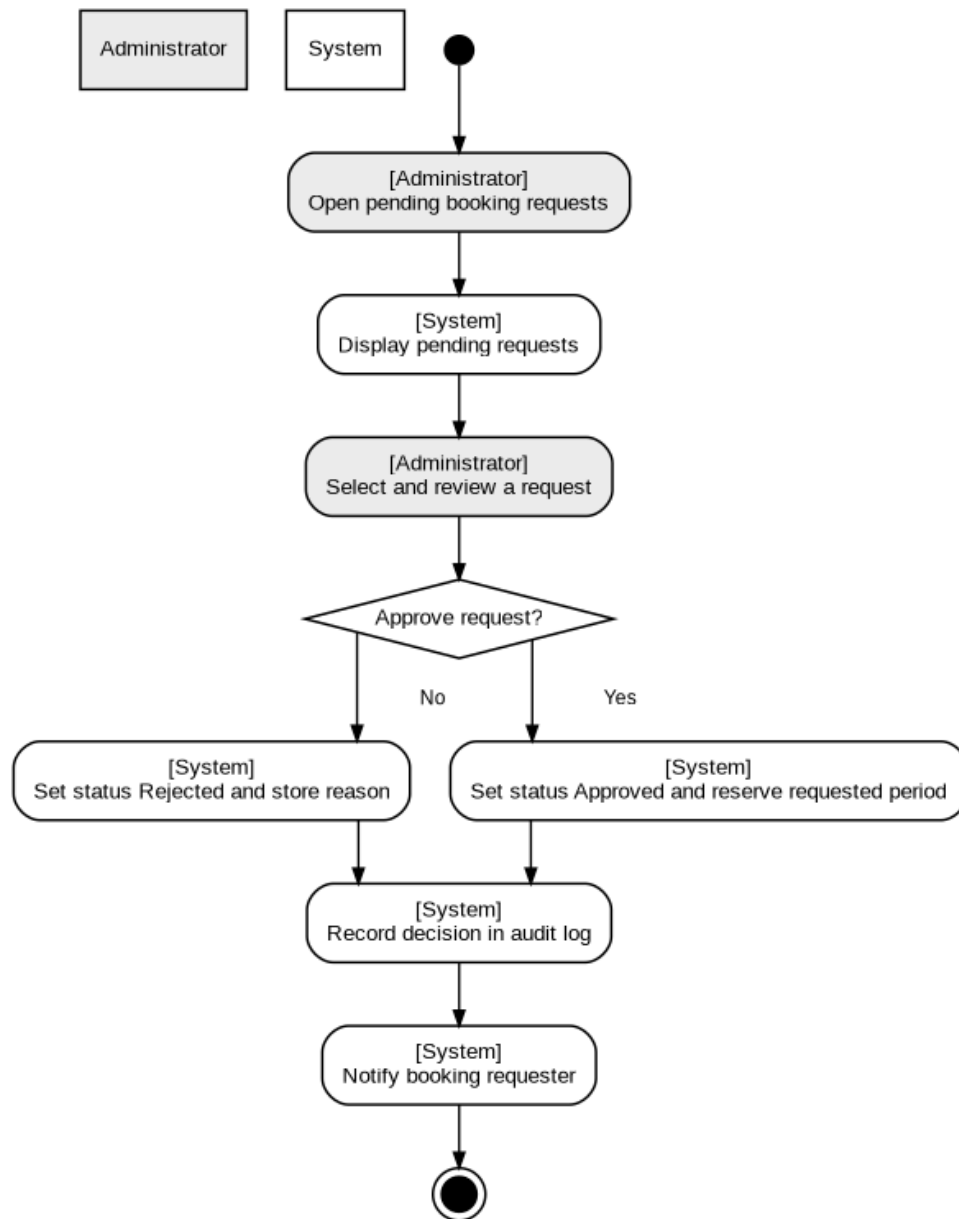
AD-4: Administrator Manages Lab Resources

Traced requirements: SRS-10 and SRS-13.



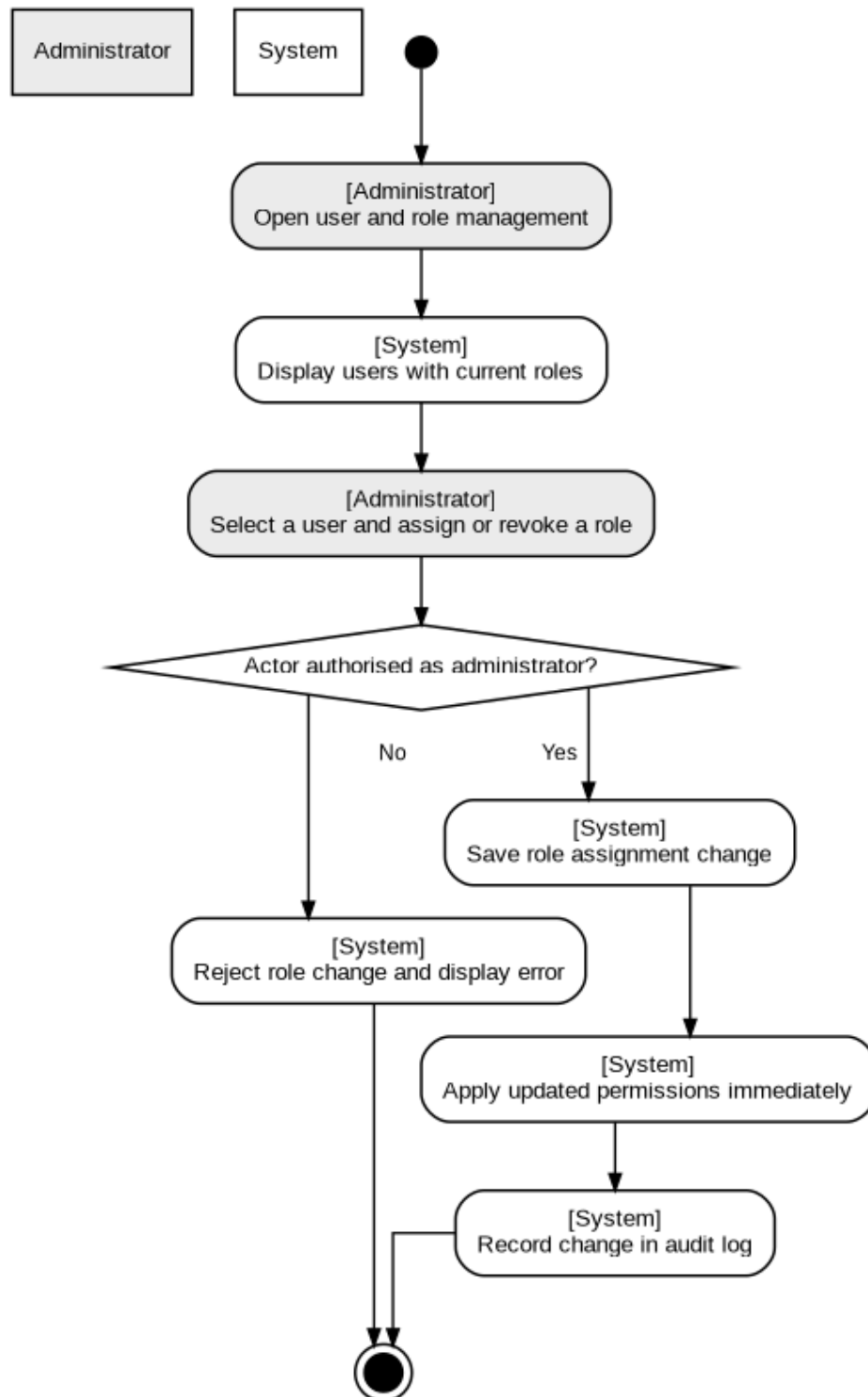
AD-5: Administrator Reviews a Pending Booking Request

Traced requirements: SRS-6, SRS-7, and SRS-13.



AD-6: Administrator Manages Roles and Permissions

Traced requirements: SRS-11, SRS-12, and SRS-13.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall display current non-archived resources to lab members and support search by resource name, filtering by category and availability status, and sorting by name, category, or availability status.
SRS-2	The system shall display the selected resource's description, location, responsible person, availability, and current status.
SRS-3	The system shall require a resource, start time, and end time for each booking request; reject the request when the resource is unavailable or overlaps an approved booking; and display the reason when the request is invalid.
SRS-4	The system shall save a valid booking request with the requester and an initial status of Pending.
SRS-5	The system shall show each lab member only their own bookings and requests with current status, allow cancellation before the booking start time, and release the reserved period when a booking is cancelled.
SRS-6	The system shall allow only an authorised administrator to review a pending booking request by approving or rejecting it; approval reserves the requested period and rejection stores the administrator's reason.
SRS-7	The system shall notify the booking requester when the status of their booking request changes.
SRS-8	The system shall require an affected resource, problem type, and description for each issue report; save the report with reporter, submission time, and Pending status; and notify the administrator.
SRS-9	The system shall allow only an authorised administrator to resolve an issue report by changing its status and adding a resolution note, and shall notify the reporter when the status changes.
SRS-10	The system shall allow only an authorised administrator to create, edit, archive, and set availability for resource records; reject records missing a valid name, category, or location; hide archived resources from normal member views; and keep archived records visible to administrators.
SRS-11	The system shall allow only an authorised administrator to assign or revoke a user role and shall apply the updated permissions immediately after the role change is saved.
SRS-12	The system shall authenticate users using Google OAuth 2.0 before protected functions are accessed and shall enforce permissions according to the authenticated user's assigned role.
SRS-13	The system shall record the acting user, timestamp, action, and affected record for changes to resource, booking, issue-report, and role records.
SRS-14	The system shall allow only an authorised administrator to view change history in chronological order.

ID	System Requirement
SRS-15	The system shall allow only an authorised administrator to view administrative records for resources, bookings, issue reports, user roles, and audit logs.

9. Software Architecture

The Lab Data Management System uses a Modular Monolithic Architecture with a Layered Client-Server structure. The system is implemented as a web application in which the frontend and backend are separated by an API interface, while backend modules operate within a single application and share a central MySQL database.

Selected technology stack:

- Frontend: React
- Backend: Node.js
- Database: MySQL
- Authentication: Google OAuth 2.0
- Development / Version Control: GitHub
- Containerisation / Deployment Packaging: Docker

Rationale

A modular monolith is suitable for the current scope because resource management, booking, issue reporting, role management, authentication, and audit logging are closely related but can still be separated into functional modules. This avoids the operational complexity of microservices while preserving clear responsibility boundaries and allowing individual modules to be changed without redesigning the entire system.

The React frontend is responsible for presentation and user interaction. The Node.js backend processes requests, validates input, enforces business rules and authorisation, coordinates functional modules, and communicates with the MySQL database. Google OAuth 2.0 authenticates users before protected functions are accessed; the backend then applies permissions from the user's assigned role.

Architecture Overview

Main communication flow: User -> React Frontend -> Node.js Backend -> Functional Module -> MySQL Database. Docker containers package the application runtime and dependencies so the development and testing environment can be reproduced consistently.

Module Interaction

When a user performs an action through the React frontend, the request is sent to the Node.js backend. The backend authenticates the user when required, applies role-based authorisation, and delegates the request to the relevant functional module. Valid changes are stored in MySQL and auditable changes are also recorded in the audit log. Status changes that require user awareness trigger the corresponding booking or issue-report notification.

Component	Layer / Type	Description
React Web Client	Presentation Layer	Presents resources, bookings, issue reports, administrative records, and system responses; accepts search/filter/sort, booking, and permitted administrative actions.
Node.js Backend	Application Layer	Receives API requests, validates input, coordinates modules, enforces business rules, and communicates with MySQL.
Authentication Module	Security / Cross-cutting	Integrates with Google OAuth 2.0 and establishes authenticated user identity before protected functions are accessed.
Authorisation / Role Management Module	Security / Business Logic	Checks assigned roles, restricts administrative functions, and applies role changes immediately.
Resource Management Module	Business Logic	Handles viewing, details, searching, filtering, sorting, create/edit/archive, validation, and availability management for resources.
Booking Management Module	Business Logic	Handles booking requests, conflict validation, member cancellation, and administrator approval/rejection.
Issue Report Module	Business Logic	Handles issue-report submission, administrator review/resolution, status updates, and reporter notifications.
Audit Log Module	Business Logic / Cross-cutting	Records acting user, timestamp, action, and affected record for auditable changes and supports chronological review.
MySQL Database	Data Layer	Stores users, roles, resources, bookings, issue reports, and audit-log records.
GitHub Repository	Development / Version Control	Stores source code and documentation and supports collaboration, version tracking, and change management.
Docker Containers	Deployment Packaging / Containerisation	Package application runtime and dependencies for consistent development, testing, and later deployment packaging.

10. Data Storage Technology

The selected data storage technology is MySQL, a Relational Database Management System (RDBMS), used as the primary persistent data store for the Lab Data Management System.

Rationale

The data defined by the SRS is structured and relational. Core entities include users and roles, resources, booking requests, issue reports, and audit-log entries. These entities have explicit relationships and consistency rules: bookings reference a user and resource, issue reports reference an affected resource, and role assignments determine permissions available to authenticated users.

MySQL supports relational tables, primary and foreign keys, constraints, indexing, and transactional operations. Transactions help preserve consistency when booking decisions or cancellations are saved, resource availability is changed, issue reports are resolved, roles are updated, and audit records are written. Indexes also support the required search, filter, sort, and administrative retrieval operations.

Component	Data Storage Technology	Description
Lab Management Database	MySQL	Stores related records for users and roles, resources, bookings, issue reports, and audit logs. Keys and constraints maintain integrity; indexes support search, filtering, sorting, and record retrieval.

11. Development Environment

The recommended development environment is a local development machine with GitHub for version control/collaboration and Docker for a reproducible application runtime used during development and testing.

Rationale

GitHub provides a central repository for source code, documentation, commits, and branches, allowing multiple team members to work on different parts of the system while retaining a traceable project history. Each team member develops and tests locally before synchronising changes through GitHub.

Docker provides a consistent runtime and dependency package across team members and provides the packaging basis for later deployment. Production deployment infrastructure remains outside the current iteration; Docker is included to standardise development/testing and deployment packaging.

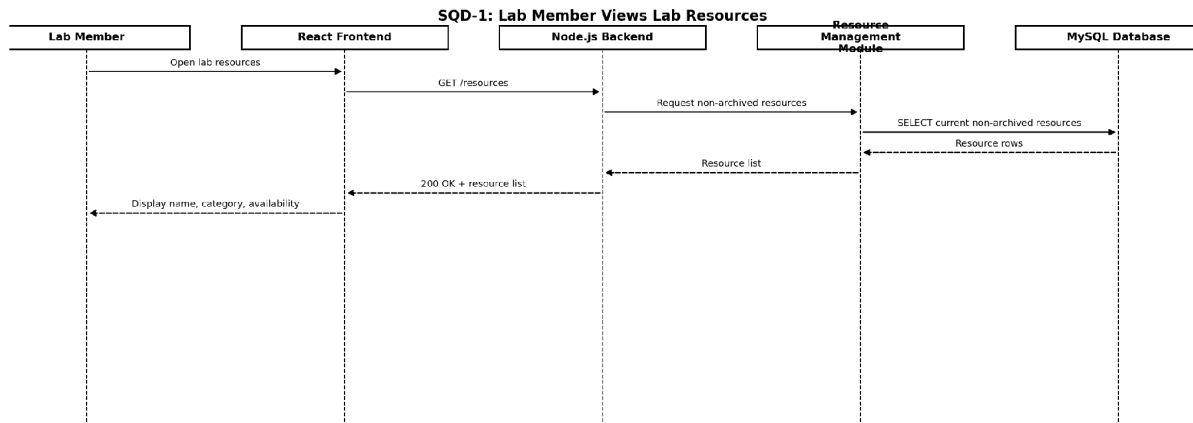
Component	Development Environment	Description
Application / System	Local Development Machine	Used to develop, run, and test resource management, booking, issue reporting, role management, authentication, and audit functionality.
Source Code & Version Control	GitHub Repository	Centralised source and documentation repository with commits, branches, reviews, and project history.
Team Development & Collaboration	GitHub + Local Development Tools	Team members work locally and synchronise changes through GitHub; branches and commits help reduce conflicts and track modifications.
Containerised Runtime	Docker	Defines a reproducible application environment for development/testing and provides deployment packaging for the application runtime.

12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the Modular Monolithic Architecture with a Layered Client-Server structure described in Section 9. Together they cover the functional interactions represented by SRS-1 through SRS-38. Each diagram identifies the current SRS requirements it traces.

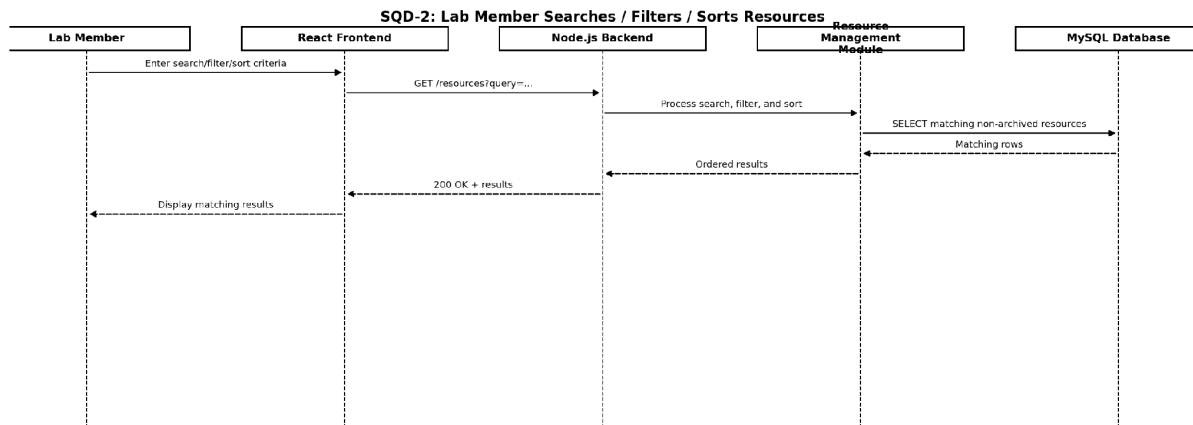
SQD-1: Lab Member Views Lab Resources

Traced requirements: SRS-1 and SRS-2.



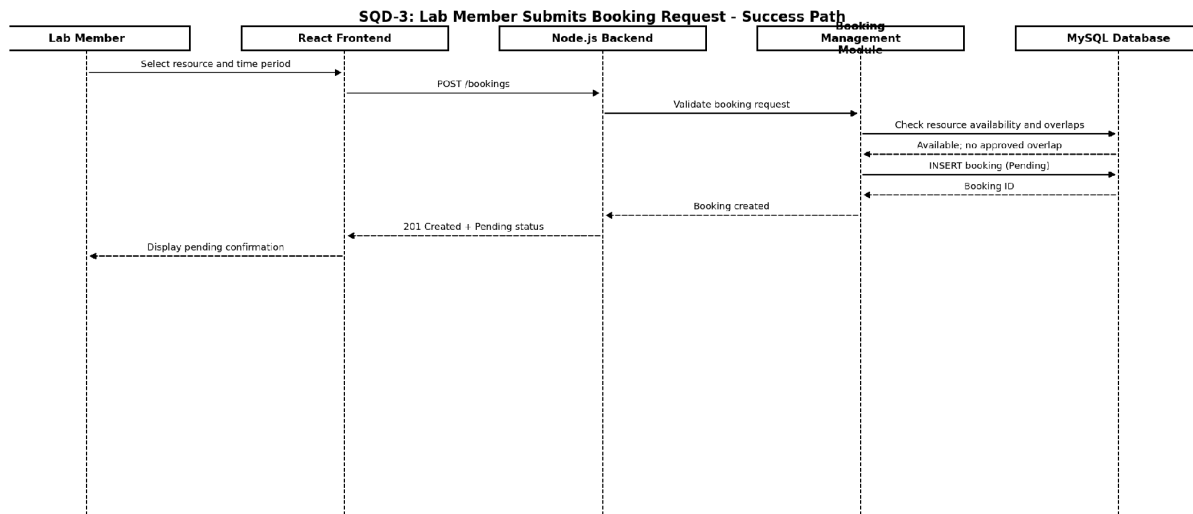
SQD-2: Lab Member Finds Resources (Search / Filter / Sort)

Traced requirements: SRS-1.



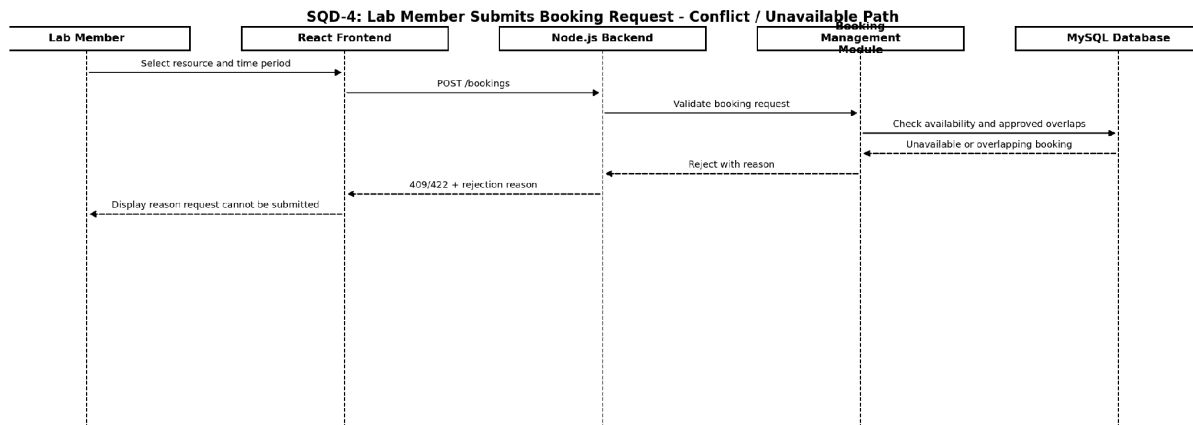
SQD-3: Lab Member Submits Booking Request - Success Path

Traced requirements: SRS-3 and SRS-4.



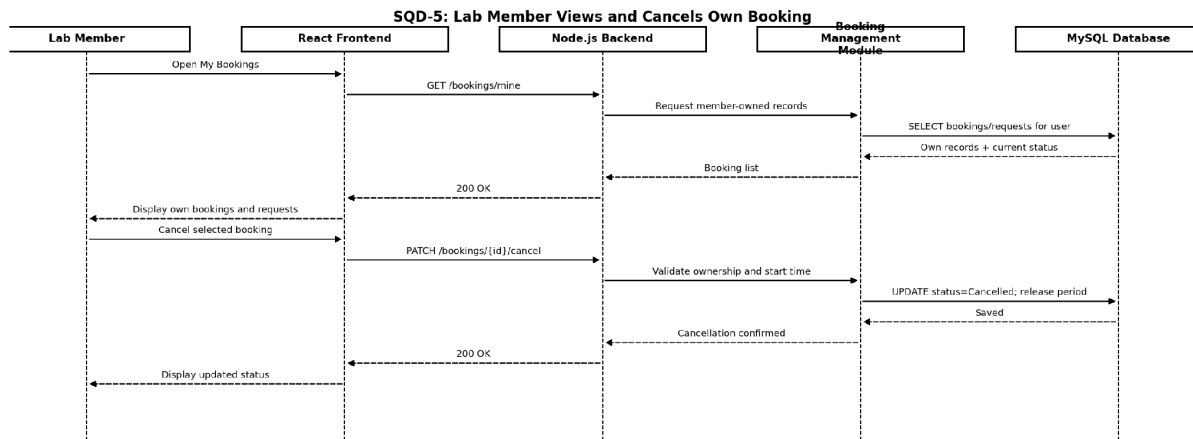
SQD-4: Lab Member Submits Booking Request - Conflict / Unavailable Path

Traced requirements: SRS-3.



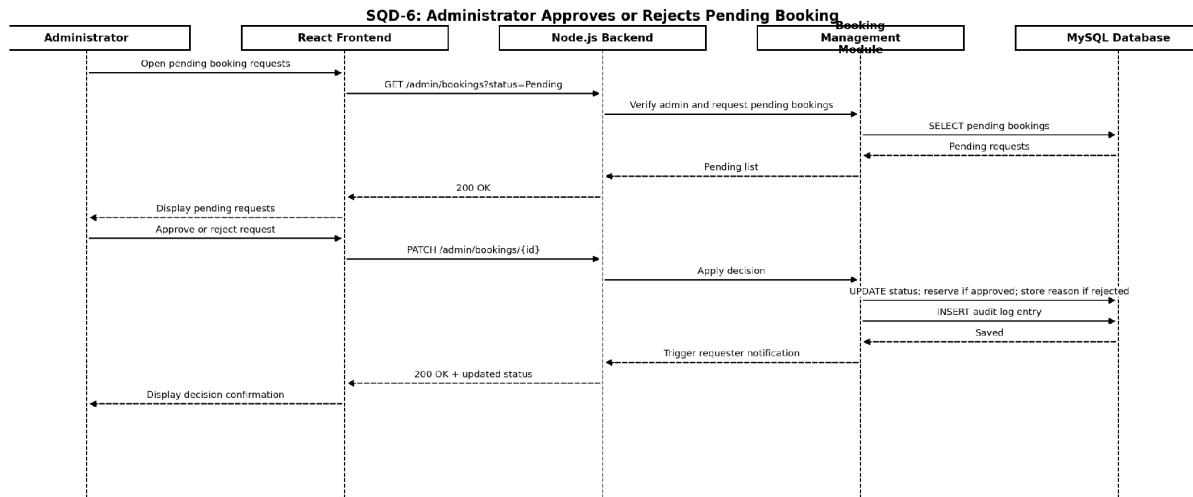
SQD-5: Lab Member Views and Cancels Own Booking

Traced requirements: SRS-5.



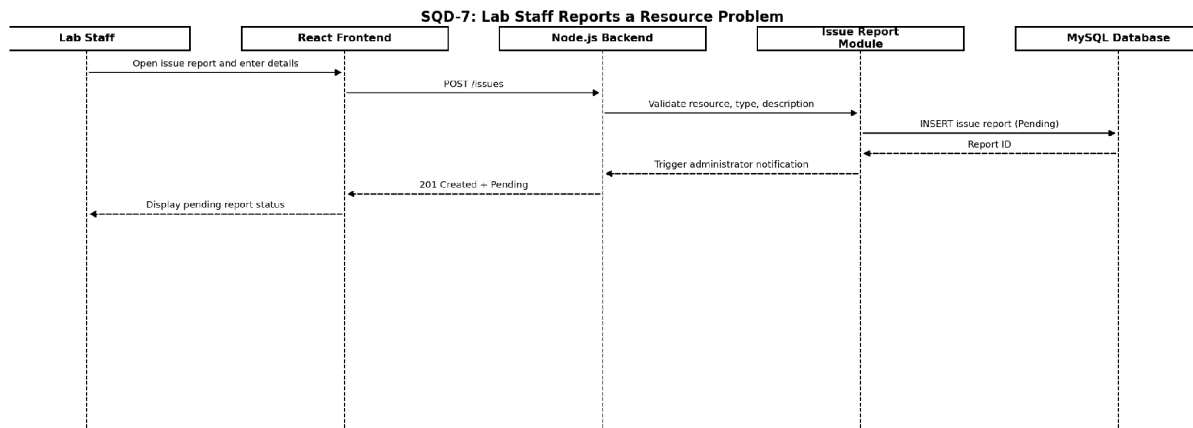
SQD-6: Administrator Approves or Rejects Pending Booking

Traced requirements: SRS-6, SRS-7, and SRS-13.



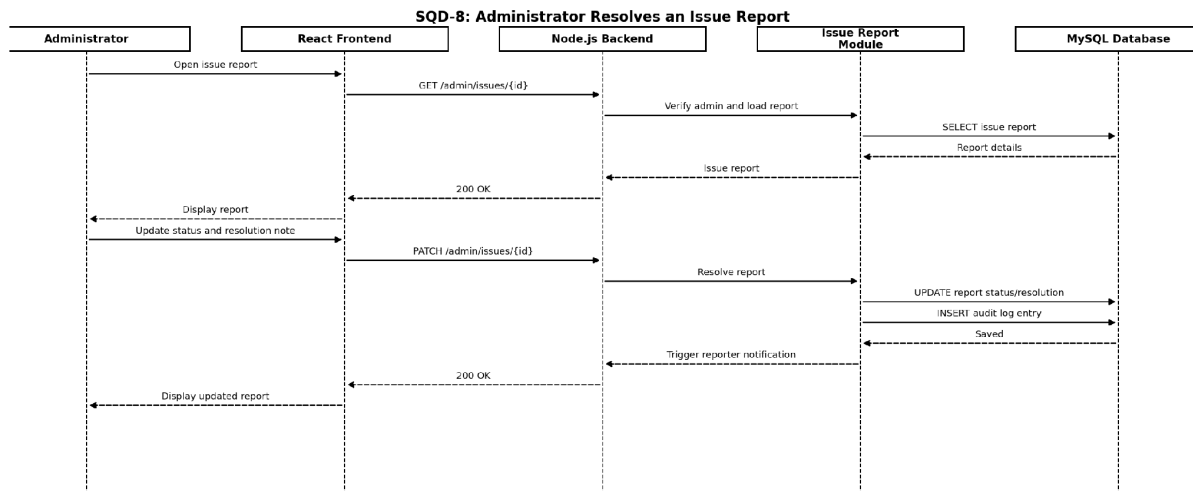
SQD-7: Lab Staff Reports a Resource Problem

Traced requirements: SRS-8



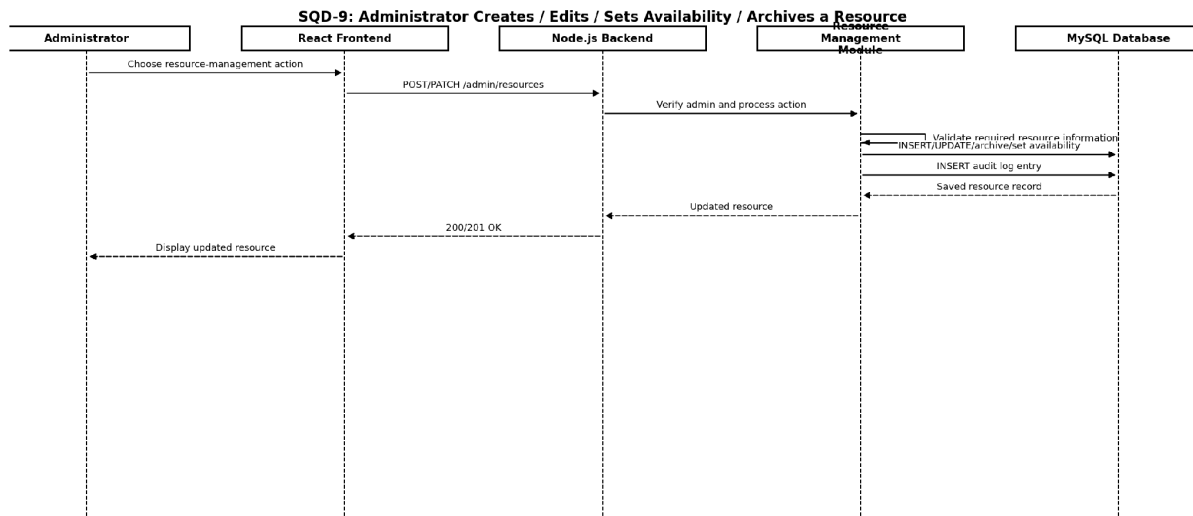
SQD-8: Administrator Resolves an Issue Report

Traced requirements: SRS-9 and SRS-13.



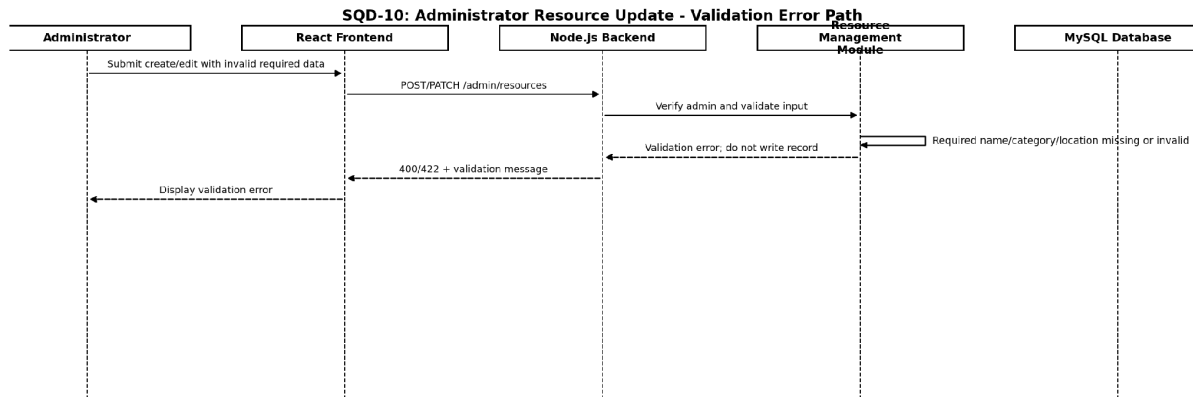
SQD-9: Administrator Creates / Edits / Sets Availability / Archives a Resource

Traced requirements: SRS-10 and SRS-13.



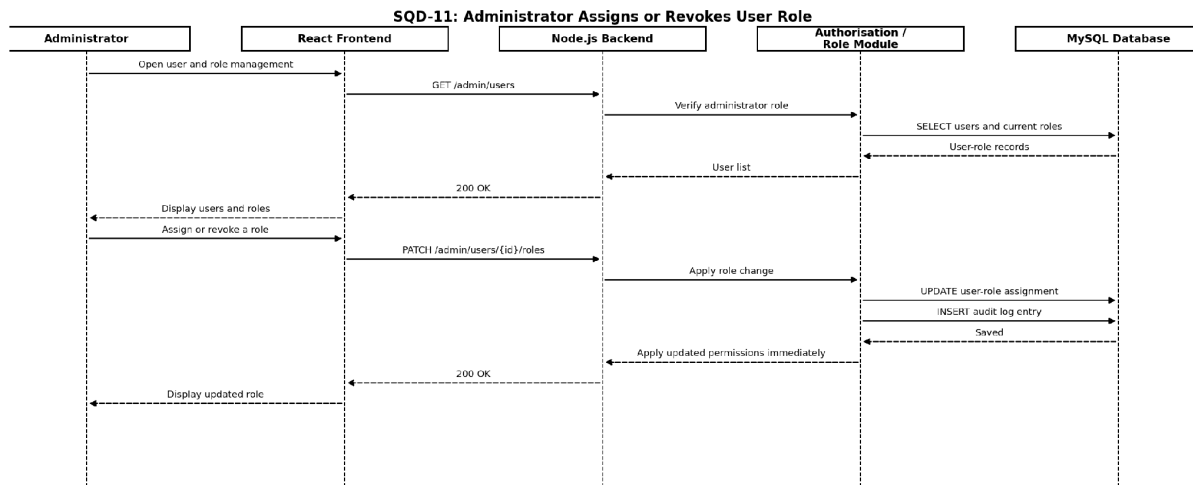
SQD-10: Administrator Resource Update - Validation Error Path

Traced requirements: SRS-10.



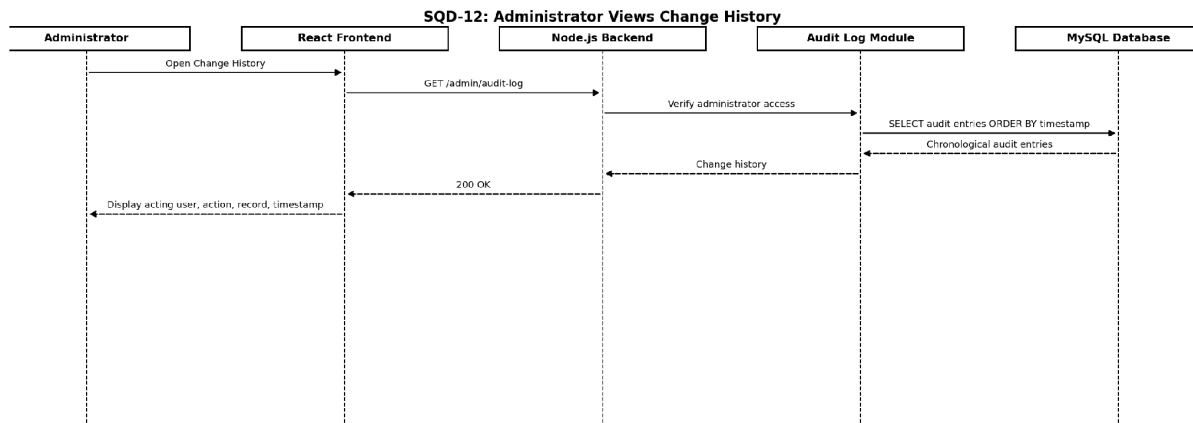
SQD-11: Administrator Assigns or Revokes User Role

Traced requirements: SRS-11, SRS-12, and SRS-13.



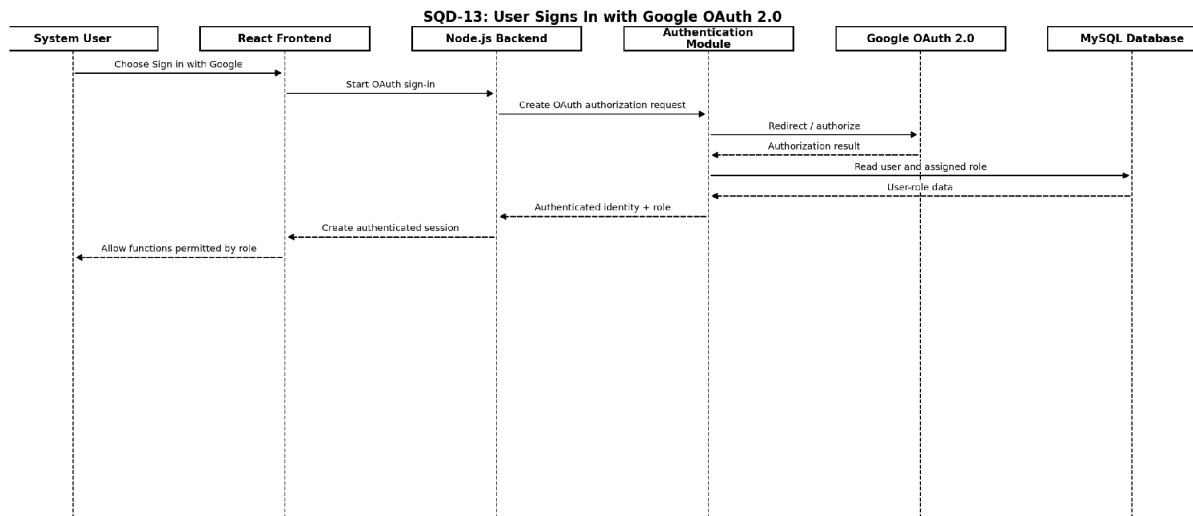
SQD-12: Administrator Views Change History

Traced requirements: SRS-14.



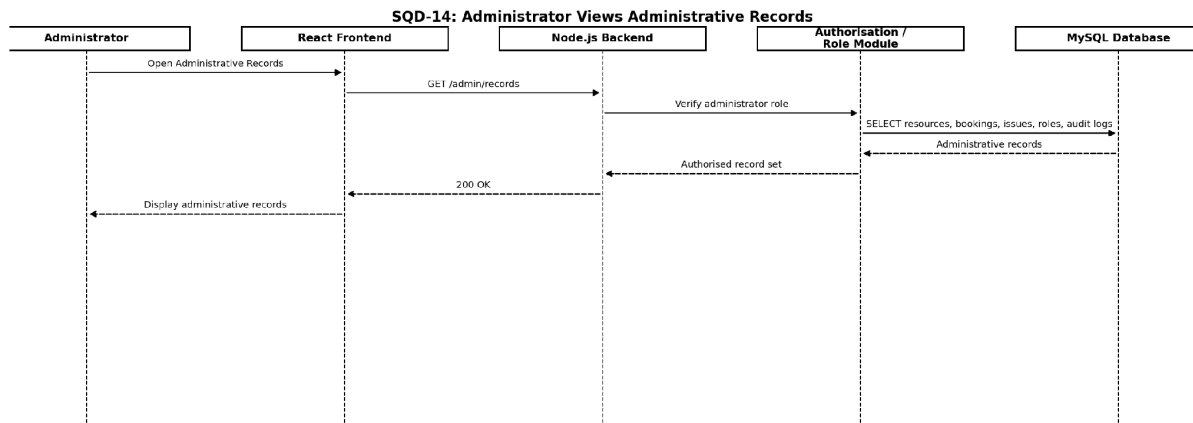
SQD-13: User Signs In with Google OAuth 2.0

Traced requirements: SRS-12.



SQD-14: Administrator Views Administrative Records

Traced requirements: SRS-15



13. Traceability Matrix

The matrix maps each user-facing use case or major action to its user story, consolidated user requirement, relevant activity diagram (when applicable), and system requirements.

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 View Lab Resources	US-1	URS-1	AD-1	SRS-1
SG-1	UC-2 Find Resources (Search / Filter / Sort)	US-2	URS-1	-	SRS-1
SG-1	UC-3 View Resource Details	US-3	URS-2	-	SRS-2
SG-1	UC-4 Submit Booking Request	US-4	URS-3	AD-1	SRS-3, SRS-4
SG-1	UC-5 View My Bookings & Requests	US-5	URS-4	AD-2	SRS-5
SG-1	UC-5 Manage My Bookings - Cancel	US-6	URS-4	AD-2	SRS-5
SG-2	UC-6 Report Issue / Data Correction	US-7	URS-5	AD-3	SRS-8
SG-3	UC-7 Manage Lab Resources - Create	US-8	URS-6	AD-4	SRS-10
SG-3	UC-7 Manage Lab Resources - Edit	US-9	URS-6	AD-4	SRS-10
SG-3	UC-7 Manage Lab Resources - Archive	US-10	URS-6	AD-4	SRS-10
SG-3	UC-7 Manage Lab Resources - Availability	US-11	URS-6	AD-4	SRS-10
SG-3	UC-8 Review Booking Requests	US-12	URS-7	AD-5	SRS-6, SRS-7
SG-3	UC-9 Resolve Issue Reports	US-13	URS-8	AD-3	SRS-9
SG-4	UC-10 Manage Roles & Permissions	US-14	URS-9	AD-6	SRS-11, SRS-12
SG-5	UC-11 View Change History / Audit	US-15	URS-10	AD-3, AD-4, AD-5, AD-6	SRS-13, SRS-14
SG-4	UC-12 Sign in with Google	US-16	URS-11	-	SRS-12
SG-3	UC-13 View Administrative Records	US-17	URS-12	-	SRS-15